

SQRT tricks

Abner Vidal

Contenidos

- Block decomposition.
- Heavy-light splitting.
- Structure rebuilding.
- Algoritmo de Mo.
- Mo con updates.
- Strings
- Algoritmos randomizados (Birthday paradox).

Block decomposition

Block decomposition es una técnica que permite responder range queries y realizar point updates, e incluso range updates en algunos casos, de forma similar a estructuras como **segment tree**, aunque con una implementación más sencilla.

La idea consiste en dividir un arreglo de tamaño N en aproximadamente $\frac{N}{S}$ bloques, cada uno de tamaño S . Para cada bloque se mantiene información agregada, como por ejemplo la suma de sus elementos.

Block decomposition

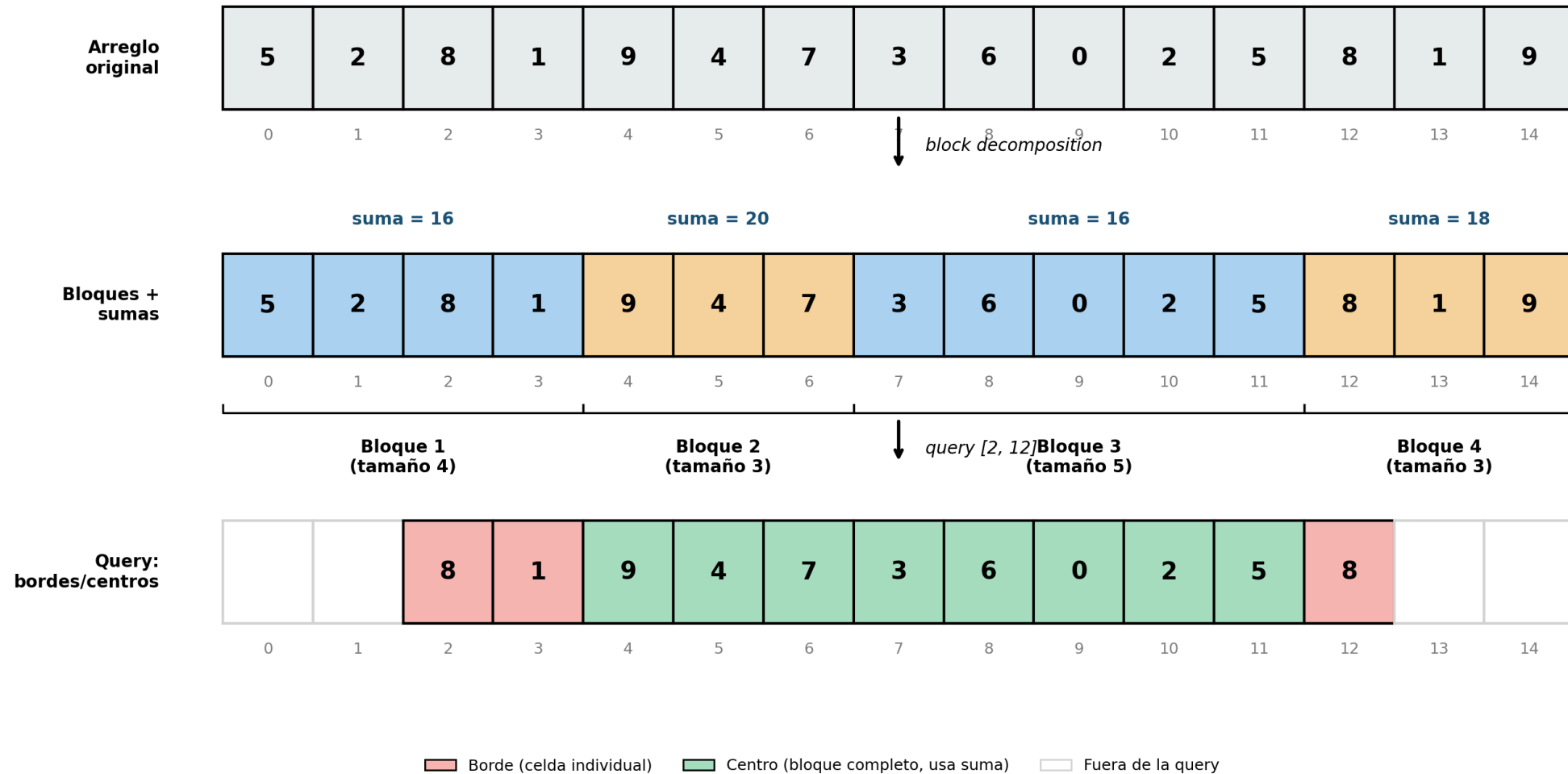
Al responder una consulta sobre un rango, solo es necesario recorrer de forma individual los elementos que pertenecen a los bloques parciales en ambos extremos del rango (a lo más S elementos por extremo). Los bloques completamente contenidos en el rango pueden procesarse utilizando la información previamente almacenada, sin recorrer sus elementos uno a uno.

De esta forma, cada consulta tiene una complejidad temporal de

$$O\left(\frac{N}{S} + S\right)$$

ya que, en el peor caso, se procesan $\frac{N}{S}$ bloques completos y hasta $2S$ elementos correspondientes a los extremos del rango.

Ejemplo



Como elegir el tamaño de S

Definimos entonces nuestra función de costo de una query:

$$f(S) = \frac{N}{S} + S$$

Como se trata de un problema de optimización entonces podemos aplicar la regla de la derivada:

$$f'(S) = 1 - \frac{N}{S^2}$$

Igualándola a 0:

$$0 = 1 - \frac{N}{S^2}$$

$$N = S^2$$

$$S = \sqrt{N}$$

El óptimo de esta función se alcanza con $S = \sqrt{N}$, pero ojo: **no siempre** el óptimo es $\sqrt{N}$. Puede que a la función de costo se le agregue un logaritmo debido al uso de búsqueda binaria o alguna estructura de datos interna, y la complejidad por query sea $O(\frac{N}{S} + S \log N)$.

En este caso, se puede demostrar que el tamaño de bloque óptimo es $S = \sqrt{N \log N}$.

Implementación

```
template <class T, T merge(T,T)>
struct sqrt_decomp {
    int n, sq;
    vector<T> st, bl;
    sqrt_decomp(vector<T> & a){
        n = a.size(); st.resize(n);
        sq = sqrtl(n); bl.resize((n+sq-1)/sq);
        for (int i = 0; i < n; i++){
            st[i] = a[i];
            bl[i/sq] = ((i%sq == 0)?st[i]:merge(bl[i/sq], st[i]));
        }
    }
    void update(int pos, T val) {
        st[pos] = val;
        int block = pos/sq, lb = block*sq, rb = min(block*sq+sq-1, n-1);
        T vl;
        for (int i = lb; i <= rb; i++)
            vl = ((i == lb)?st[i]:merge(vl, st[i]));
        bl[block] = vl;
    }
    T query(int l, int r){
        T res; bool hasR = 0;
        for (int i = l/sq; i <= r/sq; i++){
            int lb = i*sq, rb = i*sq+sq-1;
            if (l <= lb && rb <= r){
                res = (hasR?merge(res, bl[i]):bl[i]), hasR = 1;
                continue;
            }
            for (int j = max(l, lb); j <= min(rb, r); j++)
                res = (hasR?merge(res, st[j]):st[j]), hasR = 1;
        }
        return res;
    }
};
```

Ejercicios

- Dynamic Range Sum Queries
- Range Update Queries

Heavy-light splitting

Más que un algoritmo, el heavy-light splitting es un concepto: consiste en separar los objetos en dos categorías, pesados y livianos. Los objetos pesados son aquellos costosos de procesar individualmente, pero como son pocos, ese costo se compensa. Los objetos livianos, en cambio, son numerosos pero baratos de procesar, por lo que se abordan con una estrategia distinta, pensada para aprovechar justamente que hay muchos de ellos.

Aplicación

Supongamos que tenemos que realizar este tipo de operaciones sobre un arreglo:

- $query(l, r, k)$: Encontrar la suma de los valores en las posiciones $l, l + k, l + 2k, \dots$, sin superar r .

Aquí un ejemplo de una query:



■ posiciones incluidas en $query(l=2, r=11, k=3)$

El problema

La solución naive puede llegar a costar tiempo cuadrático en el peor caso.

Sin embargo, para las queries con $k > \sqrt{N}$:

- El número de saltos de tamaño k dentro del arreglo está acotado por $N/k < \sqrt{N}$.
- Por lo tanto, el costo de la naive es solo $O(\sqrt{N})$ por query.
- Para este tipo de queries, sí podemos permitirnos usar el enfoque naive.

La solución

Ya vimos que la naive nos alcanza para $k > \sqrt{N}$. Falta resolver el resto de los casos, donde $k \leq \sqrt{N}$.

El truco: como el número de valores posibles de k en este rango es a lo más $\sqrt{N}$, podemos precalcular el prefix sum de cada residuo módulo k , para cada uno de esos valores de k .

- Guardar los prefix sums para un k fijo cuesta $O(N)$.
- Hay $O(\sqrt{N})$ valores posibles de k .
- Costo total del precálculo: $O(N\sqrt{N})$.
- Cada query con $k \leq \sqrt{N}$ se responde en $O(1)$, consultando el prefix sum correspondiente.

Sin darnos cuenta, aplicamos *heavy-light splitting*:

- **Heavy (pocos, costosos de precomputar):** queries con $k \leq \sqrt{N}$. Hay a lo más $\sqrt{N}$ valores distintos de k , así que podemos precomputar sus prefix sums de una vez ($O(N\sqrt{N})$ total) y responder cada query en $O(1)$.
- **Light (muchos, baratos de procesar):** queries con $k > \sqrt{N}$. Son numerosos (todos los k posibles por sobre $\sqrt{N}$), pero cada uno se resuelve directamente con la naive en $O(\sqrt{N})$, sin necesidad de precómputo.

Structure rebuilding

Imagina un problema donde las queries se pueden resolver fácilmente gracias a un precómputo, como un prefix sum. El problema es que también hay actualizaciones, y no podemos darnos el lujo de reconstruir el precómputo completo cada vez que ocurre una...

¿O podemos?



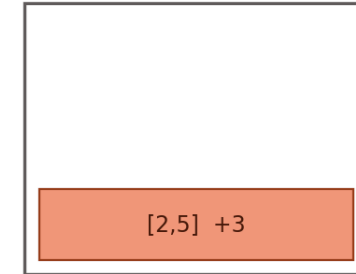
¿Qué tal si, en lugar de reconstruir el precómputo en cada actualización, lo hacemos cada $O(\sqrt{N})$ actualizaciones? Esto resuelve el problema del costo de reconstrucción, pero introduce uno nuevo: cada query ahora debe considerar también las actualizaciones ocurridas desde el último precómputo, ya que estas todavía no están reflejadas en él. Como a lo más hay $O(\sqrt{N})$ actualizaciones pendientes en esa ventana, cada query se ve afectada por este mismo factor $O(\sqrt{N})$ adicional.

Ejemplo

Update(2,5,+3) -> se agrega a la cola (1/3)

0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

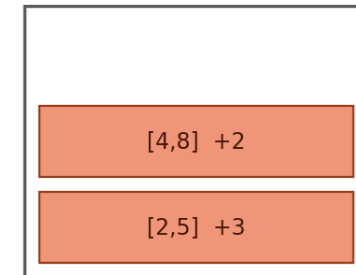
cola de updates



Update(4,8,+2) -> se agrega a la cola (2/3)

0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

cola de updates



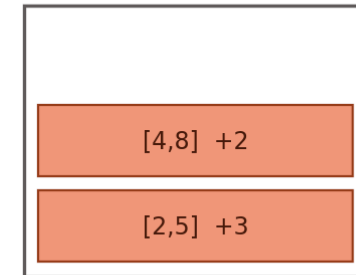
Ejemplo

Query(5)



$$\text{query}(5) = \text{base}[5] = 0 + 3 + 2 = 5$$

cola de updates



Ejemplo

Update(1,3,+4) -> Rebuild

Update(1,3,+4) -> cola llena (3/3)

0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

cola de updates

[1,3] +4
[4,8] +2
[2,5] +3

Rebuild: se aplica todo al arreglo base y se vacia la cola

4	7	7	5	5	2	2	2	0	0
---	---	---	---	---	---	---	---	---	---

cola de updates

vacia

Ejemplo

Query(3)

4	7	7	5	5	2	2	2	0	0
---	---	---	---	---	---	---	---	---	---

$\text{query}(3) = \text{base}[3] = 7 = 7$

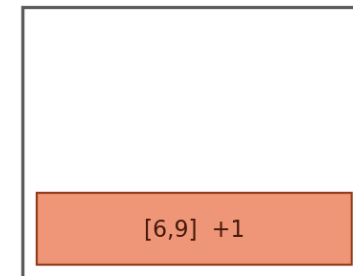
Update(6,9,+1) -> se agrega a la cola (1/3)

4	7	7	5	5	2	2	2	0	0
---	---	---	---	---	---	---	---	---	---

cola de updates



cola de updates



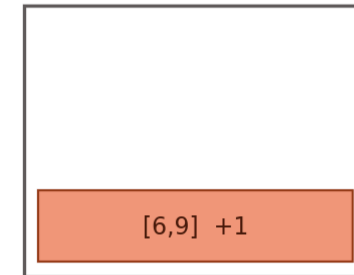
Ejemplo

Query(7)

4	7	7	5	5	2	2	2	0	0
---	---	---	---	---	---	---	---	---	---

$\text{query}(7) = \text{base}[7] = 2 + 1 = 3$

cola de updates



Implementación

```
int n,q;
cin >> n >> q;
int sq = sqrt(n);
vector<array<int,3>> upd;
vector<ll> ans(n,0);
while (1){
    int op;
    cin >> op;
    if (op == 1){ // update
        int l,r,val;
        cin >> l >> r >> val, l--, r--;
        upd.push_back({l,r,val});
        if (upd.size() == sq){
            vector<ll> ev(n,0);
            for (auto [l,r,val] : upd) {
                ev[l] += val;
                if (r+1 < n) ev[r+1] -= val;
            }
            for (int i = 1; i < n; i++) ev[i] += ev[i-1];
            for (int i = 0; i < n; i++) ans[i] += ev[i];
            upd.clear();
        }
    } else { // query
        int pos;
        cin >> pos, pos--;
        ll res = ans[pos];
        for (auto [l,r,val] : upd){
            if (l <= pos && pos <= r){
                res += val;
            }
        }
        cout << res << endl;
    }
}
```

Algoritmo de Mo

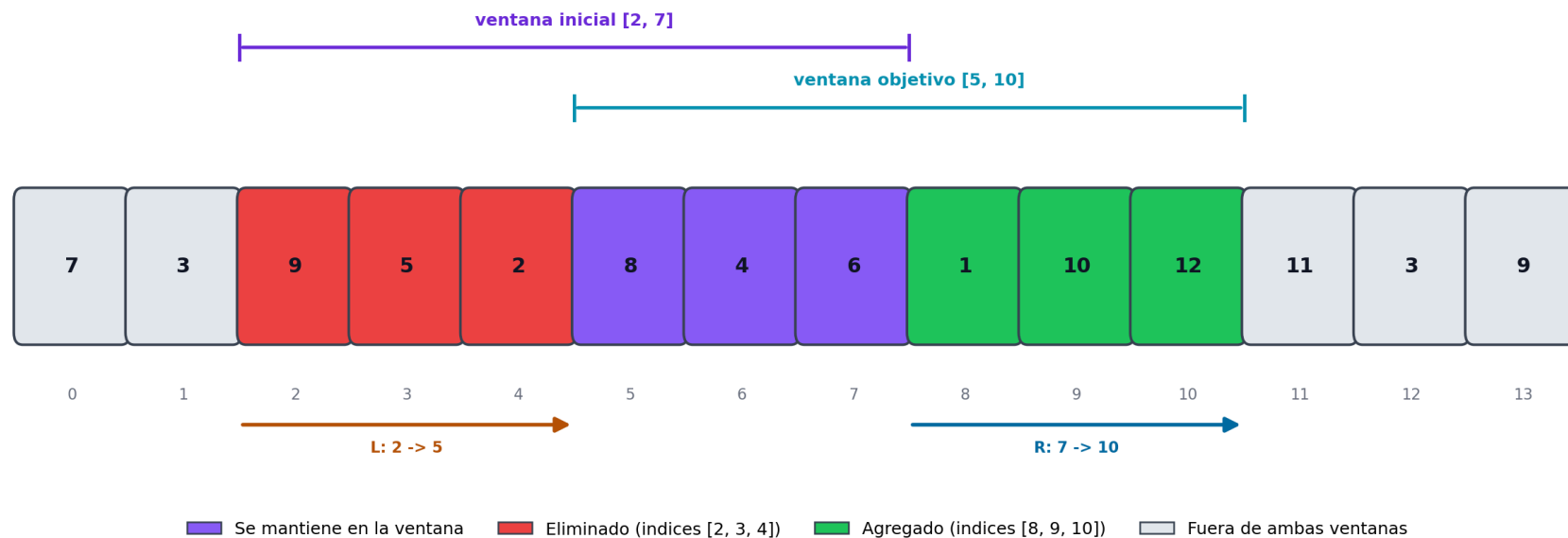
Supongamos que tenemos un arreglo de tamaño N , y tenemos que responder el siguiente tipo de consulta:

- $query(l, r)$: responder cuantos elementos distintos hay en el rango $[l, r]$.

Una forma naive de responder cada $query(l, r)$ es recorrer el subarreglo $[l, r]$ y contar los elementos distintos con un set o un arreglo de frecuencias, lo cual cuesta $O(r - l + 1)$ por consulta.

De una query a otra

Pero por un instante que tal si en lugar de borrar completamente la respuesta de una query a otra, ¿cual sería el costo de transformar la respuesta de un rango en la respuesta de otro rango?:



Evidentemente, si realizamos las queries en el orden que se nos da, podríamos terminar con una complejidad cuadrática.

Ahí es donde entra la magia del algoritmo de Mo: aprovechándonos de que no existen actualizaciones dentro del arreglo, podemos realizar el algoritmo de forma offline, y realizar las queries en un orden conveniente.

Criterio de ordenamiento(Basado en block decomposition):

1. Agrupar por bloque de l : $\text{block} = \lfloor l/B \rfloor$
2. Si se encuentran dentro del mismo bloque, ordenar por r .

Analicemos por separado el movimiento de cada puntero: $curL$ y $curR$.

Usamos bloques de tamaño $B = \sqrt{N}$, así que hay $\frac{N}{B} = \sqrt{N}$ bloques.

Movimiento del puntero L

Al moverse de una query a otra existen solo dos casos para el puntero L : ambas queries pertenecen a un mismo bloque, o a bloques adyacentes. En ambos casos:

- Como L varía a lo más $O(B)$, moverse entre dos queries consecutivas del mismo bloque cuesta **a lo más** B pasos.
- Si hay Q queries en total, el costo de L dentro de los bloques es:

$$O(Q \cdot B) = O(Q\sqrt{N})$$

Movimiento del puntero R

Dentro de un mismo bloque, r está ordenado, por lo tanto R **se mueve monótonamente**, sin retrocesos, mientras estemos en ese bloque.

- El recorrido total de R dentro de un bloque está acotado por el tamaño del arreglo: $O(N)$.
- Como hay $\sqrt{N}$ bloques:

$$O(N) \times \sqrt{N} = O(N\sqrt{N})$$

Sumando todo

$$\underbrace{O(Q\sqrt{N})}_{\text{mov. de } L} + \underbrace{O(N\sqrt{N})}_{\text{mov. de } R} = O((N + Q)\sqrt{N})$$

Intuición clave:

- L se mueve *poco* por query, pero *muchas* veces.
- R se mueve *mucho* en total, pero *pocas* veces.
- Elegir $B = \sqrt{N}$ **equilibra ambos términos**, minimizando la suma.

Implementación

```
struct query {
    ll l, r, i;
};
template<class T, typename U, class T2>
struct mo_algorithm {
    vector<U> ans;
    template<typename... Args>
    mo_algorithm(vector<T> &v, vector<query> &queries, Args... args) {
        T2 ds(args...);
        ll block_sz = sqrtl(v.size());
        ans.assign(queries.size(), -1);
        sort(queries.begin(), queries.end(), [&](auto &a, auto &b) {
            return a.l/block_sz != b.l/block_sz ? a.l/block_sz < b.l/block_sz : a.r < b.r;
        });
        ll l = 0, r = -1;
        for (query q : queries) {
            while (l > q.l) ds.add(v[--l]);
            while (r < q.r) ds.add(v[++r]);
            while (l < q.l) ds.remove(v[l++]);
            while (r > q.r) ds.remove(v[r--]);
            ans[q.i] = ds.answer(q);
        }
    }
};
```

Problemas de práctica

- CSES - Distinct Values Queries
- Codeforces 86D - Powerful array

Mo con updates

¿Qué pasa si el arreglo permite **updates** (además de queries)?

Ya no podemos ordenar solo por (block_l, r) : una query puede ser válida *antes* de cierta actualización pero *inválida* después (o viceversa), porque el contenido del arreglo cambia con el tiempo.

Idea: agregar una tercera dimensión — el **tiempo** t , entendido como "*cuántas actualizaciones han ocurrido hasta el momento en que se realiza esta query*".

Cada consulta deja de ser un par (l, r) y pasa a ser (l, r, t) .

Nuevo criterio de ordenamiento

Usamos bloques de tamaño B (ahora en **dos** dimensiones: l y t):

1. Agrupar por bloque de l : $\text{block}_l = \lfloor l/B \rfloor$
2. Dentro del mismo bloque de l , agrupar por bloque de t : $\text{block}_t = \lfloor t/B \rfloor$
3. Dentro del mismo par de bloques, ordenar por r

Procesamiento

Además de mover $curL$ y $curR$ como antes, ahora mantenemos un puntero $curT$ que representa **cuántas actualizaciones ya aplicamos**.

Para pasar de una query a la siguiente:

- Movemos $curL$ y $curR$ agregando/quitando elementos (igual que en Mo clásico).
- Movemos $curT$ hacia t :
 - Si $curT < t$: **aplicamos** la siguiente actualización (y si el índice actualizado cae dentro de $[curL, curR]$, ajustamos `cnt` / `distinct`).
 - Si $curT > t$: **revertimos** la última actualización aplicada (deshaciendo su efecto sobre `cnt` / `distinct` si corresponde).

Cada actualización debe guardar tanto el valor **nuevo** como el **anterior**, para poder aplicarse y revertirse en $O(1)$.

Complejidad

Con tres punteros moviéndose sobre un espacio de tamaño N dividido en bloques de tamaño B , el análisis (idéntico en espíritu al caso clásico) da un costo total de:

$$O\left(\frac{Q \cdot N}{B} + Q \cdot B + N \cdot B\right)$$

Optimizando sobre B se obtiene $B = N^{2/3}$, y por lo tanto:

$$O(N^{5/3})$$

Intuición: ahora tenemos *tres* recursos que balancear (L, R, T) en vez de dos, así que el bloque óptimo ya no es $\sqrt{N}$ sino $N^{2/3}$ — el mismo principio de "equilibrar costos", aplicado a una dimensión extra.

Lema: Si tenemos un conjunto de strings, de forma que la suma de sus largos es S , entonces existen a lo más $O(\sqrt{S})$ largos diferentes de strings.

Demostración:

Dividimos los largos en dos casos:

- Hay a lo más $\sqrt{S}$ largos diferentes hasta $\sqrt{S}$.
- Hay a lo más $\sqrt{S}$ largos diferentes mayores a $\sqrt{S}$, para no superar S .

Problema: Se nos da un set de strings, que sus largos suman a lo más S , y el objetivo es encontrar un par, de forma que uno es substring del otro.

Solución: Debido a que existen a lo más $\sqrt{S}$ largos diferentes, podemos probar con hashing en cada posición si encaja o no con otro del mismo largo.

Esto es debido a que solo debemos verificar $\sqrt{S}$ largos diferentes por cada posición, y por lo tanto la complejidad es de $O(S\sqrt{S})$.

Paradoja del cumpleaños

¿Cuántas personas se requieren para tener una probabilidad del 50% de que hayan dos con el mismo cumpleaños?

Respuesta

En un grupo de **23 personas** hay más de 50% de probabilidad de que dos compartan cumpleaños.

Con N valores posibles (365 días), la probabilidad de colisión al elegir k elementos al azar crece rápido:

$$P(c) \approx 1 - e^{-k^2/2N}$$

Punto clave: basta tomar $k \approx \sqrt{N}$ muestras para tener alta probabilidad de colisión, no N .

Algoritmo baby-step giant-step (BSGS)

Resuelve $g^x = h \pmod{p}$, con x desconocido, en un grupo de orden N .

Se escribe $x = i \cdot m + j$, con $m = \lceil \sqrt{N} \rceil$, $0 \leq i, j < m$.

Baby steps: tabla de $h \cdot g^{-j}$ para $j = 0, \dots, m - 1$

Giant steps: calcular g^{im} para $i = 0, \dots, m - 1$ y buscar coincidencia con la tabla

Cuando $g^{im} = h \cdot g^{-j}$, se obtiene $x = im + j$

Por qué es una colisión tipo cumpleaños

Ambas listas (baby y giant steps) tienen tamaño $m \approx \sqrt{N}$, tomadas del mismo espacio de tamaño N .

Buscar una coincidencia entre dos listas de tamaño $\sqrt{N}$ es exactamente el escenario del paradoja del cumpleaños: se necesita ese orden de muestras para que la colisión sea casi segura.

Por eso BSGS logra tiempo y memoria $O(\sqrt{N})$, en vez de $O(N)$ de la fuerza bruta.